GE VERNOVA

DERMS Performance Test Framework

1.0.0

User Guide

Copyright

Copyright 2025, GE Vernova and/or its affiliates ("GE Vernova"). All Rights Reserved.

GE and the GE Monogram are trademarks of General Electric Company used under trademark license.

This document is the confidential and proprietary information of GE Vernova and may not be reproduced, transmitted, stored, or copied in whole or in part, or used to furnish information to others, without the prior written permission of GE Vernova.

Contents

1. Getting Started with DERMS Performance Test Framework ... 7
1.1. Quick Start Guide ... 7
1.1.1. Step 1: Understanding the Framework ... 7
1.1.2. Step 2: Configure Your Deployment ... 7
1.1.3. Step 3: Set Up Test Scenarios ... 8
1.1.4. Step 4: Deploy the Framework ... 8
1.1.5. Step 5: Analyze Generated Test Data ... 8
1.1.6. Step 6: Monitor and Analyze ... 9
1.2. Additional Resources ... 9
1.2.1. Quick Reference ... 9
1.2.2. Prerequisites Checklist ... 9
1.2.3. Need Help? ... 10
1.3. What's Next? ... 10
2. DERMS Performance Test Framework - Environment Variables ... 11
2.1. Helm Values ... 11
2.1.1. Protocol and Connection Settings ... 11
2.1.2. Configuration Files ... 11
2.1.3. Certificate Configuration ... 11
2.1.4. File Processing ... 12
2.2. Kafka Environment Variables ... 12
2.3. Zitadel Authentication Environment Variables ... 12
2.4. Locust Python Runner Environment Variables ... 13
2.5. Telemetry Environment Variables ... 13
2.6. Locust Load Testing Environment Variables ... 13
2.6.1. Basic Configuration ... 13
2.6.2. Load Test Parameters ... 13
2.6.3. Network Configuration ... 14
2.6.4. Test Configuration ... 14
2.7. Configuration Notes ... 14
2.7.1. Security Considerations ... 14
2.7.2. Resource Management ... 14
2.7.3. Authentication ... 15
2.8. Customization ... 15
2.9. Related Files ... 15
3. DERMS Performance Test Framework Modules Configuration Files Guide ... 16
3.1. Overview ... 16
3.1.1. ConfigMap ... 16
3.2. Configuration Files Structure ... 16
3.3. Configuration Files Overview ... 16
3.3.1. main.yaml ... 16

Proprietary - See Copyright Page

3.3.2. gateway.yaml 17
3.3.2.1. Gateway Configuration 17
3.3.3. operations.yaml 19
3.3.4. data_generation.yaml 19
3.3.4.1. DER Ratings Templates 23
3.3.4.2. Network Model Structure 24
3.3.4.3. File Generation Settings 24
3.3.4.4. Generation Modes 24
3.4. Configuration Best Practices 24
3.4.1. File Organization 24
3.4.2. Configuration Management 25
3.4.3. Performance Tuning 25
3.4.4. Testing Strategy 25
3.5. Troubleshooting Common Issues 25
3.5.1. Configuration Validation 25
3.5.2. Test Execution Issues 26
3.5.3. Performance Optimization 26
3.6. Conclusion 26
4. DERMS Performance Test Framework - Deployment Guide 27
4.1. Prerequisites 27
4.2. Post-Deployment Steps on DERMS Cluster 27
4.2.1. Network Port Configuration 27
4.2.2. RBAC Policy Updates 27
4.2.3. Gateway-Proxy Environment Variables 28
4.2.4. LWTS Manager Configuration 28
4.2.5. Download Kafka CA Certificate 28
4.2.6. Collect Module Client Credentials 28
4.3. PTF Deployment 29
4.3.1. Create Persistent Volume Directories 29
4.3.2. Connect to PTF Cluster 29
4.3.3. Verify Cluster Connection 29
4.3.4. Create Namespace for PTF 30
4.3.5. Create Kafka CA Secret 30
4.3.6. Create Configuration ConfigMap 30
4.3.7. Verify ConfigMap Creation 30
4.3.8. Deploy PTF Helm Chart 30
4.4. Values File Configuration 30
4.5. Verification Steps 32
4.5.1. Pod Status 32
4.5.2. Service Status 32
4.5.3. ConfigMap and Secrets 32
4.5.4. Logs 32

Proprietary - See Copyright Page

4.6. Troubleshooting 32
4.6.1. Common Issues 32
4.7. Security Considerations 33
4.8. Related Documentation 33
5. Data Generation Service - Accessing Generated Data 34
5.1. Overview 34
5.2. Prerequisites 34
5.3. Data Storage Locations 34
5.4. Accessing Master Node Data 34
5.4.1. Step 1: Connect to Master Node 34
5.4.2. Step 2: Navigate to Master Data Directory 35
5.4.3. Step 3: List Generated Files 35
5.5. Accessing Worker Node Data 35
5.5.1. Step 1: Navigate to Worker Data Directory 35
5.5.2. Step 2: List Generated Worker Directories 35
5.5.3. Step 3: Access Individual Worker Data 36
5.6. Data Types and Formats 36
5.7. Copying Data Files 36
5.7.1. Copy Files to Local Machine 36
5.8. Best Practices 37
5.8.1. Data Access 37
5.8.2. Security 37
5.9. Integration with PTF Workflows 37
6. Log Analysis in Headless Mode 38
6.1. Overview 38
6.2. Prerequisites 38
6.3. Accessing Kubernetes Worker Pods 38
6.3.1. Step 1: List PTF Worker Pods 38
6.3.2. Step 2: Connect to Worker Pod 38
6.4. Accessing Log Files 39
6.4.1. Navigate to Logs Directory 39
6.4.2. List Available Log Files 39
6.5. Log Analysis Commands 39
6.5.1. Viewing Log Contents 39
6.5.2. Searching Log Contents 40
6.5.3. Filtering and Analysis 40
6.6. Copying Logs for External Analysis 40
6.7. Log Analysis Best Practices 41
6.7.1. Performance Analysis 41
6.7.2. Error Investigation 41
6.7.3. Test Execution Analysis 41
6.8. Common Log Patterns and Solutions 41

Proprietary - See Copyright Page

6.9. Monitoring Multiple Pods 42
6.10. Troubleshooting 42
6.10.1. Pod Access Issues 42
6.10.2. Missing Log Files 42
6.11. Additional Resources 42
7. Locust Web UI Guide for DERMS Performance Test Framework 44
7.1. Overview 44
7.2. Prerequisites 44
7.3. Configuration 44
7.3.1. Required Configuration Values 44
7.3.1.1. Locust Web Authentication 44
7.3.1.2. Ingress Configuration 45
7.3.1.3. Locust Mode Configuration 45
7.4. Deployment 45
7.4.1. Update Values File 45
7.4.2. Deploy with Helm 45
7.4.3. Verify Deployment 45
7.5. Accessing the Web UI 46
7.5.1. Access URL 46
7.5.2. Login 46
7.6. Using the Web UI 47
7.6.1. Dashboard Overview 47
7.6.2. Starting a Load Test 47
7.6.3. Monitoring Test Results 48
7.6.3.1. Statistics Tab 48
7.6.3.2. Charts Tab 49
7.6.3.3. Failures Tab 50
7.6.4. Download Test Data 50
7.6.5. Exceptions PTF 50
7.6.6. Load Test Logs 51
7.6.7. Worker Status 51
7.7. Troubleshooting 52
7.7.1. Common Issues 52
7.7.1.1. Web UI Not Accessible 52
7.7.2. Authentication Issues 52
7.7.3. Workers Not Connecting 53
7.7.4. Debug Mode 53
7.8. Best Practices 53
7.8.1. Security 53
7.8.2. Performance 53
7.8.3. Testing Strategy 53
7.9. Conclusion 54

Proprietary - See Copyright Page

1. Getting Started with DERMS Performance Test Framework

The DERMS Performance Test Framework (PTF) is a comprehensive testing solution designed to evaluate the performance of DERMS (Distributed Energy Resource Management System) components under various load conditions. This framework helps you simulate real-world scenarios and measure system performance metrics.

1.1. Quick Start Guide

To quickly get started with the DERMS Performance Test Framework, follow this recommended reading order:

1.1.1. Step 1: Understanding the Framework

Start Here: DERMS Performance Test Framework Framework Introduction

Begin by understanding what the PTF framework is, its architecture, and core concepts. This foundational knowledge will help you navigate the rest of the documentation effectively.

What you'll learn:

- PTF framework overview and architecture.
- Key components and their roles.
- Testing capabilities and use cases.

1.1.2. Step 2: Configure Your Deployment

Next: Environment Variables and Deployment Configuration

Learn about the various deployment configurations available and understand the environment variables that control PTF behavior.

What you'll learn:

- Helm values configuration.
- Environment variables and their purposes.
- Protocol and connection settings.
- Service configuration options.

Proprietary - See Copyright Page

1.1.3. Step 3: Set Up Test Scenarios

Then: Modules Configuration

Configure your test scenarios by setting up the DERMS module configuration files. This step defines what and how you want to test.

What you'll learn:

- Configuration file structure (`main.yaml`, `gateway.yaml`, `operations.yaml`, `data_generation.yaml`).
- Load curve modeling for different DERMS modules.
- Test scenario definition.
- ConfigMap creation for Kubernetes deployment.

1.1.4. Step 4: Deploy the Framework

Install: Deployment Guide

Follow the step-by-step installation guide to deploy DERMS Performance Test Framework in your Kubernetes environment.

What you'll learn:

- Prerequisites and requirements.
- Kubernetes deployment steps.
- Network configuration.
- RBAC policy setup.
- Post-deployment verification.

1.1.5. Step 5: Analyze Generated Test Data

Prepare Data: Data Generation Service

The DERMS Performance Test Framework includes a data generation service that creates test data for performance testing scenarios. See the above page to access the generated data files stored on Kubernetes nodes, with different locations for master and worker nodes.

What you'll learn:

- How to access generated data on cluster nodes.
- Data storage locations (master vs. worker nodes).
- File management and organization.
- Data verification steps.

Proprietary - See Copyright Page

1.1.6. Step 6: Monitor and Analyze

Final Step: Log Analysis in Headless Mode

Learn how to monitor test execution and analyze results through log analysis, especially when running in headless mode.

What you'll learn:

- Accessing Kubernetes worker pods.
- Log file locations and formats.
- Performance metrics analysis.
- Troubleshooting common issues.

1.2. Additional Resources

1.2.1. Quick Reference

Topic	Description	Time to Read
Framework Introduction	Core concepts and architecture	10 minutes
Deployment Configuration	Environment setup and variables	15 minutes
Module Configuration	Test scenario setup	20 minutes
Installation Guide	Step-by-step deployment	30 minutes
Data Generation	Test data preparation	10 minutes
Log Analysis	Results monitoring and analysis	15 minutes

1.2.2. Prerequisites Checklist

Before you begin, ensure you have:

- Access to a Kubernetes cluster.
- Admin privileges on cluster nodes.
- `kubectl` and `helm` CLI tools installed.
- Access to Artifactory registry.
- SSH access to cluster nodes.
- Basic understanding of Kubernetes concepts.

Proprietary - See Copyright Page

1.2.3. Need Help?

If you encounter issues during setup or testing:

- **Check Prerequisites:** Ensure all requirements from the checklist above are met
- **Follow the Order:** Complete each step in the recommended sequence
- **Review Logs:** Use the log analysis guide to troubleshoot deployment issues
- **Verify Configuration:** Double-check your configuration files against the examples

1.3. What's Next?

Ready to get started? Begin with the DERMS Performance Test Framework Framework Introduction to understand the core concepts, then follow the step-by-step guide above.

Each section builds upon the previous one, so following the recommended order will give you the smoothest experience with the DERMS Performance Test Framework.

Proprietary - See Copyright Page

2. DERMS Performance Test Framework - Environment Variables

This chapter describes all environment variables configured in the Helm `values.yaml` file for the DERMS Performance Test Framework.

2.1. Helm Values

2.1.1. Protocol and Connection Settings

Variable	Default Value	Description	Type
common.environment.protocol	http	Protocol used for communication (http/https)	string
common.environment.gatewayHost	<None>	Hostname or cluster DNS of the DERMS Service	string
common.environment.port	8080	Port number for the gateway UI service	integer
common.environment.apiVersionPath	api/v3	API version path for UI Service REST endpoints	string
common.environment.serviceName	ui-service	Name of the service being tested	string

2.1.2. Configuration Files

Variable	Default Value	Description	Type
common.environment.mainConfigFile	config_files/main.yaml	Path to the main configuration file	string
common.environment.dersInfoFile	der_info.csv	Name of the DERs information CSV file	string
common.environment.outputFolder	generated_files	Directory for generated output files	string
common.environment.inputFolder	input_files	Directory for input files	string

2.1.3. Certificate Configuration

Variable	Default Value	Description	Type
common.environment.certSubject	/CN=192.168.31.251, 127.0.0.1,gateway-proxy,172.18.0.19, 172.18.0.15	Certificate subject with multiple common names	string
common.environment.certCA	ca	Certificate Authority identifier	string

Proprietary - See Copyright Page

2.1.4. File Processing

Variable	Default Value	Description	Type
common.environment.measurementChunkSize	1000	Size of measurement data chunks for processing	integer
common.environment.filesToExclude	ca_cert/ca.crt, ca_cert/ca.pem, certificate/ca.crt, ertificate/ca.key, sample_cert/ca.crt, sample_cert/client.crt, sample_cert/client.key, der_info.csv	Comma-separated list of files to exclude from processing	string
common.environment.lfdiFile CreationWaitTimeSec	30	Wait time in seconds for LFDI file creation	integer

2.2. Kafka Environment Variables

Variable	Default Value	Description	Type
kafka.environment.kafkaBootstrapServer	<None>	Kafka bootstrap server master node IP of one of the k8 node	string
kafka.environment.kafkaPort	31443	Kafka Bootstrap server k8 node port	integer
kafka.environment.kafkaCaLocation	/performance-test-framework/ kafka/certs/ca.crt	Path to Kafka CA certificate file	string
kafka.environment.kafkaCaLocationDir	/performance-test-framework/kafka/certs	Directory containing Kafka certificates	string
kafka.environment.kafkaGroupId	ptf-kafka-group	Kafka consumer group identifier	string
kafka.environment.kafkaOffsetReset	earliest	Kafka offset reset strategy	string
kafka.environment.kafkaSecurityProtocol	SSL	Kafka security protocol (SSL/PLAINTEXT)	string
kafka.environment.kafkaEnableSsl	false	Enable SSL for Kafka connections	boolean

2.3. Zitadel Authentication Environment Variables

Variable	Default Value	Description	Type
zitadel.environment.useFoundationAuth	false	Enable Foundation authentication	boolean
zitadel.environment.zitadelBaseUrl	<None>	Base URL for Zitadel authentication service	string
zitadel.environment.proxy	<None>	HTTP proxy server URL	string
zitadel.environment.sslVerification	false	Enable SSL certificate verification	boolean

Proprietary - See Copyright Page

2.4. Locust Python Runner Environment Variables

Variable	Default Value	Description	Type
runner.environment.heartbeatIntervalSec	30	Interval between heartbeat signals in seconds	integer
runner.environment.heartbeatLivenessSec	90	Heartbeat liveness timeout in seconds	integer
runner.environment. masterHeartbeatTimeoutSec	300	Master heartbeat timeout in seconds	integer
runner.environment.cpuMonitorIntervalSec	60.0	CPU monitoring interval in seconds	float
runner.environment. cpuWarningThresholdPercent	100	CPU usage warning threshold percentage	integer

2.5. Telemetry Environment Variables

Variable	Default Value	Description	Type
telemetry.environment.gatewayProxyHost	master node IP of one of the k8 node	Gateway proxy hostname/IP for telemetry	string

2.6. Locust Load Testing Environment Variables

2.6.1. Basic Configuration

Variable	Default Value	Description	Type
locust.environment.headless	true	Run Locust in headless mode (without web UI)	boolean
locust.environment.locustHost	<None>	Target DERMS host for load testing	string
locust.environment.locustMasterHost	locust-master	Hostname of the Locust master node	string
locust.environment.locustMode	distributed	Locust execution mode (single/distributed)	string
locust.environment.locustExpectWorkers	3	Expected number of Locust worker nodes	integer

2.6.2. Load Test Parameters

Variable	Default Value	Description	Type
locust.environment.runTimeDuration	15m	Duration for the load test run	string

Proprietary - See Copyright Page

Variable	Default Value	Description	Type
locust.environment.ackWorkerTimeout	20	Worker acknowledgment timeout in seconds	integer
locust.environment.maxRetries	5	Maximum number of retry attempts	integer
locust.environment.locustUsers	120	Total number of simulated users	integer
locust.environment.locustSpawnRate	1	Rate at which users are spawned per second	integer
locust.environment.batchSize	100	Batch size for processing requests	integer
locust.environment.workerWaitTime	300	Worker wait time in seconds	integer

2.6.3. Network Configuration

Variable	Default Value	Description	Type
locust.environment.locustWebUiPort	1005	Port for Locust web UI	integer
locust.environment.locustCommPort	5557	Port for Locust master-worker communication	integer

2.6.4. Test Configuration

Variable	Default Value	Description	Type
locust.environment.locustTestFile	locust_new/tasks/	Path to Locust test files	string
locust.environment.loadShapeClass	locust_new/load_shapes/custom_load.py	Path to custom load shape class	string

2.7. Configuration Notes

2.7.1. Security Considerations

- Many sensitive values like passwords, client secrets, and usernames are left empty in the default configuration.
- SSL verification is disabled by default (`zitadel.environment.sslVerification: false`).
- Kafka SSL is disabled by default (`kafka.environment.kafkaEnableSsl: false`).

2.7.2. Resource Management

- CPU monitoring is set to 100% threshold by default.

Proprietary - See Copyright Page

- Measurement chunk size is set to 1000 for optimal processing.

2.7.3. Authentication

- Zitadel authentication is configured, but Foundation authentication is disabled by default.
- Web authentication for Locust UI is available, but requires manual configuration.

2.8. Customization

To customize these environment variables for your deployment:

1. Create a custom `values.yaml` file or use Helm's `--set` flag.
2. Override specific values as needed for your environment.
3. Ensure all required secret values are properly configured before deployment.
4. Review network connectivity settings for your specific infrastructure.

2.9. Related Files

- `helm/values.yaml`: Main Helm values file.

Proprietary - See Copyright Page

3. DERMS Performance Test Framework Modules Configuration Files Guide

3.1. Overview

This chapter provides a comprehensive guide to model load curve shape for each DERMS module and mount them in the DERMS Performance Test Framework k8 cluster as `configMap`. These files define the core application settings, test scenarios, and data generation parameters for the performance testing framework.

As defined installation guide, these configuration files must be defined in file directory `config_files/` to be mounted as `configMap` in the DERMS Performance Test Framework k8 cluster.

3.1.1. ConfigMap

```
kubectl create configmap ptf-config --from-file=<config_files_directory_path>  --namespace <ptf-
namespace>
```

3.2. Configuration Files Structure

The `config_files/` directory contains four main YAML configuration files:

- `main.yaml`: Main orchestration and global settings.
- `gateway.yaml`: Gateway performance test configuration.
- `operations.yaml`: Operations testing configuration.
- `data_generation.yaml`: DER data and network topology generation settings.

3.3. Configuration Files Overview

3.3.1. main.yaml

The main configuration file that orchestrates other configuration files and sets global Locust settings.

```yaml
file-paths:
  gateway-config: config_files/gateway.yaml
  operation-config: config_files/operations.yaml
  data-gen-config: config_files/data_generation.yaml
```


Proprietary - See Copyright Page

```yaml
locust-settings:
  mode: distributed
```

Key Properties:

- `file-paths`: References to other configuration files
- `locust-settings.mode`: Locust execution mode

3.3.2. gateway.yaml

Configuration for gateway-related performance tests and API endpoints.

3.3.2.1. Gateway Configuration

```yaml
main:
  enable: true
  tasks:
    rpc:
      wait_time_sec: 15
      batch_size: 50
      asset_life_cycle_wait_time_sec: 20
    program:
      wait_time_sec: 10
      total_per_worker: 5
      supported_program_type: FSA2
      ders_per_program: 10
    measurement_ui:
      wait_time_sec: 15
    dispatch:
      wait_time_sec: 20
      total_users: 1
      control_type: "kW % Rated Capacity"
      control_value: 2
      control_event_offset_min: 10
      control_event_duration_min: 60
      enable_dispatch: true
      controls_per_dispatch: 10
    bulk_curtailment:
      wait_time_sec: 75
      control_event_offset_min: 10
      control_event_duration_min: 60
      enable_bulk_curtailment: true
      controls_per_dispatch: 800
    ieee:
      active_power_wait_time_sec: 30
      reactive_power_wait_time_sec: 30
      voltage_power_wait_time_sec: 60
      der_status_wait_time_sec: 60
      der_polling_wait_time_sec: 60
```

```
    der_discovery:
      wait_time_sec: 5
    kafka:
      der_commissioning_wait_time_sec: 3
      latency_wait_time_sec: 3
  reports:
    csv_format: true
    json_format: true
  port:
    registration_service: 8075
    ui_service: 8080
    ieee_service: 8444
    lwts_service: 8079
    dispatch_scheduler_service: 8073
  node_port:
    ieee_service: 30444
    kafka_service: 31443
```

Task Categories:

- **RPC Tasks**: Registration, Provision and Commissioning of DER Assets
  - `wait_time_sec`: Wait time between each Sequential RPC User task.
  - `batch_size`: Number of total DERs to process in one batch.
  - `asset_life_cycle_wait_time_sec`: DER and Aggregator Stage Lifecycle wait time.
- **Program Tasks**: FSA2 Program and Enrollment User Task
  - `total_per_worker`: Total programs per worker.
  - `supported_program_type`: Program type (FSA2).
  - `ders_per_program`: Number of DERs per program.
- **Dispatch Tasks**: Control dispatch User Task
  - `control_type`: Type of control command.
  - `control_value`: Control value to apply.
  - `control_event_offset_min`: Offset time for control events.
  - `control_event_duration_min`: Duration of control events.
  - `controls_per_dispatch`: Number of controls per dispatch operation.
- **IEEE 2030.5 Tasks**: DER telemetry User Task
  - `active_power_wait_time_sec`: Wait time for active power of a DER
  - `reactive_power_wait_time_sec`: wait time for reactive power of a DER
  - `voltage_power_wait_time_sec`: wait time for voltage of a DER
  - `der_status_wait_time_sec`: wait time for status of a DER
  - `der_polling_wait_time_sec`: wait time for polling of a DER APIs
- **Docker and Node Port Details**:

- `port`: field only applicable for docker development setup. Defines internal service ports for various DERMS services.
- `node_port`: field only applicable for k8 cluster setup.

No Changes required for the above fields.

3.3.3. operations.yaml

Configuration for operations-related performance tests (default: disabled).

```yaml
main:
  enable: false
  reports:
    csv_format: true
    json_format: true
  tasks:
    chart:
      create_wait_time_sec: 180
      edit_wait_time_sec: 120
      get_wait_time_sec: 60
      concurrent_users: 2
      total_charts: 5
    group:
      create_wait_time_sec: 180
      edit_wait_time_sec: 120
      get_wait_time_sec: 60
      concurrent_users: 2
      total_groups: 20
    table:
      create_wait_time_sec: 180
      edit_wait_time_sec: 60
      get_wait_time_sec: 60
      concurrent_users: 3
```

Task Categories: set `enable: true` to activate operation load testing.

- **chart**: Dashboard chart load testing user scenario.
- **group**: User group management load testing user scenario.
- **Table**: Data table operations load testing user scenario.

Each task type includes timing configuration and concurrency settings.

3.3.4. data_generation.yaml

Configuration for generating test data including DER (Distributed Energy Resource) definitions and network topology.

```yaml
pv_ratings: &pv_ratings
```

```
  kV: 19
  maxkW: 900
  maxkVar: 20
  maxkVarNeg: 10
  maxkVA: 800
  maxChargingkW: 3
  maxChargingkVA: 4
  minPFunderExcited: 1.95
  minPFoverExcited: 1.95
  underExcitedPF: 3.0
  overExcitedPF: 3.0
  maxDischargingkW: 6.0
  maxDischargingkVA: 0.0
  maxAh: 5.0

generic_ratings: &generic_ratings
  kV: 19
  maxkW: 900
  maxkVar: 20
  maxkVarNeg: 10
  maxkVA: 800
  maxChargingkW: 3
  maxChargingkVA: 4
  minPFunderExcited: 1.95
  minPFoverExcited: 1.95
  underExcitedPF: 3.0
  overExcitedPF: 3.0
  maxDischargingkW: 6.0
  maxDischargingkVA: 0.0
  maxAh: 5.0

wind_ratings: &wind_ratings
  kV: 19
  maxkW: 900
  maxkVar: 20
  maxkVarNeg: 10
  maxkVA: 800
  maxChargingkW: 3
  maxChargingkVA: 4
  minPFunderExcited: 1.95
  minPFoverExcited: 1.95
  underExcitedPF: 3.0
  overExcitedPF: 3.0
  maxDischargingkW: 6.0
  maxDischargingkVA: 0.0
  maxAh: 5.0

load_ratings: &load_ratings
  kV: 19
  maxkW: 900
  maxkVar: 20
  maxkVarNeg: 10
  maxkVA: 800
```

```yaml
  maxChargingkW: 3
  maxChargingkVA: 4
  minPFunderExcited: 1.95
  minPFoverExcited: 1.95
  underExcitedPF: 3.0
  overExcitedPF: 3.0
  maxDischargingkW: 6.0
  maxDischargingkVA: 0.0
  maxAh: 5.0

ders_attributes: &DERS
  count: 1
  types:
    - PV
    - Wind
  pv:
    ratings: *pv_ratings
  generic:
    ratings: *generic_ratings
  wind:
    ratings: *wind_ratings
  load:
    ratings: *load_ratings
  deru:
    count: 2
    percent: 3


lnm_model: &LNM_MODEL
  transmission_region:
    count: 1
  substations:
    count: 2
    assignment_type: equal
    transformers:
      count: 1
      feeders:
        count: 1
        aggregators:
          mrid_prefix: avfdd783
          count: 3
          ders:
            count: 1
            types:
              - PV
              - Wind
            pv:
              ratings: *pv_ratings
            generic:
              ratings: *generic_ratings
            wind:
              ratings: *wind_ratings
            load:
```

```yaml
              ratings: *load_ratings
            deru:
              count: 2
              percent: 3
            ieee_2030_count: 3
            scada_count: 3
            mrid_prefix: avfdd783
        direct-connect:
          count: 1
          types:
            - PV
            - Wind
          pv:
            ratings: *pv_ratings
          generic:
            ratings: *generic_ratings
          wind:
            ratings: *wind_ratings
          load:
            ratings: *load_ratings
          deru:
            count: 2
            percent: 3
          ieee_2030_count: 6
          scada_count: 6


lnm_model: *LNM_MODEL
input_files:
  templates: ./input_files/templates
  certs: ./certs
generated_files:
  dir: generated_files
  files:
    global_file_prefix: FI
    substation_file_prefix: partition
    aggregators_file_prefix: aggregators
    cert_dir: certificates/
    program_file_prefix: programs
    enrolment_file_prefix: enrollment
    measurement_file_prefix: measurements
mode:
  fresh_generate: true
  multiply_current_network:
    is_enabled: false
    existing_substation_file_path: input_files
    transmission_region:
      count: 1
    substations:
      count: 1
      assignment_type: equal
      transformers:
        count: 1
```

```yaml
      feeders:
        count: 1
        aggregators:
          mrid_prefix: abcd453
          count: 5
          ders:
            count: 1
            types:
              - PV
              - Wind
            pv:
              ratings: *pv_ratings
            generic:
              ratings: *generic_ratings
            wind:
              ratings: *wind_ratings
            load:
              ratings: *load_ratings
            deru:
              count: 2
              percent: 3
            ieee_2030_count: 6
            scada_count: 6
            mrid_prefix: abcd453
        direct-connect:
          count: 1
          types:
            - PV
            - Wind
          pv:
            ratings: *pv_ratings
          generic:
            ratings: *generic_ratings
          wind:
            ratings: *wind_ratings
          load:
            ratings: *load_ratings
          deru:
            count: 2
            percent: 3
          ieee_2030_count: 6
          scada_count: 6
```

3.3.4.1. DER Ratings Templates

The file defines reusable rating templates using YAML anchors:

Rating Properties:

- `kV`: Voltage rating in kilovolts.

- `maxkW`: Maximum active power in kilowatts.
- `maxkVar`/`maxkVarNeg`: Maximum reactive power (positive/negative).
- `maxkVA`: Maximum apparent power in kilovolt-amperes.
- `maxChargingkW`/`maxChargingkVA`: Maximum charging power ratings.
- `minPFunderExcited`/`minPFoverExcited`: Minimum power factor values.
- `maxDischargingkW`/`maxDischargingkVA`: Maximum discharging power ratings.
- `maxAh`: Maximum ampere-hour capacity.

3.3.4.2. Network Model Structure

Network Hierarchy:

1. **Transmission Region**: Top-level network region.
2. **Substations**: Electrical substations with transformers.
3. **Feeders**: Distribution feeders from transformers.
4. **Aggregators**: DER aggregation points.
5. **DERs**: Individual distributed energy resources.

3.3.4.3. File Generation Settings

Key Properties:

- `dir`: Output directory for generated files.
- Various prefixes for different file types.
- `cert_dir`: Directory for generated certificates.

3.3.4.4. Generation Modes

Mode Options:

- `fresh_generate`: Generate completely new network data.
- `multiply_current_network`: Scale existing network topology.

3.4. Configuration Best Practices

3.4.1. File Organization

The configuration files are organized hierarchically:

- `main.yaml` serves as the entry point and references other configuration files.
- Specific test scenarios are separated into dedicated files (`gateway.yaml`, `operations.yaml`).
- Data generation is isolated in its own configuration file for maintainability.

3.4.2. Configuration Management

- **Modular Design**: Each file focuses on a specific aspect of the testing framework.
- **Reusable Templates**: Use YAML anchors and references for common configurations.
- **Environment Flexibility**: Support for different environments through parameter adjustment.
- **Version Control**: Track configuration changes through Git for reproducibility.

3.4.3. Performance Tuning

- **Wait Times**: Adjust `wait_time_sec` values based on system response characteristics.
- **Batch Sizes**: Configure appropriate batch sizes for optimal throughput.
- **Concurrency**: Balance concurrent users with system capacity.
- **Data Scale**: Generate appropriate data volumes for realistic testing.

3.4.4. Testing Strategy

- **Incremental Testing**: Start with small loads and gradually increase.
- **Scenario Isolation**: Enable/disable specific test scenarios as needed.
- **Data Variety**: Generate diverse DER types and network topologies.
- **Monitoring**: Enable comprehensive reporting for analysis.

3.5. Troubleshooting Common Issues

3.5.1. Configuration Validation

- **YAML Syntax**: Ensure proper YAML formatting and indentation.
- **File References**: Verify all file paths are correct and accessible.
- **Parameter Values**: Check that numeric values are within expected ranges.
- **Boolean Settings**: Ensure enable/disable flags are properly set.

3.5.2. Test Execution Issues

- **Data Generation**: Verify generated files are created successfully.
- **Network Connectivity**: Check gateway host and port accessibility.
- **Authentication**: Validate OAuth2 credentials and certificates.
- **Resource Limits**: Monitor system resources during test execution.

3.5.3. Performance Optimization

- **Load Balancing**: Distribute the test load across multiple workers.
- **Memory Management**: Adjust batch sizes to prevent memory issues.
- **Network Optimization**: Configure appropriate timeout values.
- **Data Cleanup**: Manage generated files to prevent disk space issues.

3.6. Conclusion

The configuration files in the `config_files/` directory provide comprehensive control over the DERMS Performance Test Framework behavior. Understanding these configurations enables:

- **Customized Testing**: Tailor test scenarios to specific requirements.
- **Scalable Performance Testing**: Adjust parameters for different load levels.
- **Flexible Data Generation**: Create realistic network topologies and DER configurations.
- **Comprehensive Reporting**: Generate detailed analysis data for performance evaluation.

These configuration files form the foundation for effective performance testing of DERMS systems, providing the flexibility and control needed for thorough system validation.

4. DERMS Performance Test Framework - Deployment Guide

This chapter provides step-by-step instructions for deploying the DERMS Performance Test Framework (PTF) in a production environment.

4.1. Prerequisites

Before proceeding with the deployment, ensure you have:

- Access to the target Kubernetes cluster
- Admin privileges on cluster nodes
- Access to Artifactory registry
- kubectl and helm CLI tools installed
- Access to K9s (optional but recommended)

4.2. Post-Deployment Steps on DERMS Cluster

4.2.1. Network Port Configuration

Open the following ports for the master node IP (`172.31.7.180`) of the DERMS cluster on the PTF cluster:

Service	Port	Description
IEEE	30444	IEEE communication port
Kafka Bootstrap Server	30445, 31443, 30446, 30443	Kafka broker communication ports

4.2.2. RBAC Policy Updates

Remove RBAC policies from Dispatch Scheduler Service APIs for client ID `dergateway-app-admin@foundation.local`.

Remove the following `when` condition from `requestPrincipals` source:

```
when:
  - key: request.auth.claims[sws_permissions]
    values:
      - manualDispatch.monitor.write.writeAll
      - manualDispatch.schedule.write.writeAll
```


Proprietary - See Copyright Page

4.2.3. Gateway-Proxy Environment Variables

For deployments with over 10,000 DERs, increase the following environment variables in `gateway-proxy` by a multiple of 10:

Variable	Description
REQUESTS_PER_MINUTE_MUP	Requests per minute for MUP
REQUESTS_PER_MINUTE_DER	Requests per minute for DER
REQUESTS_PER_MINUTE_EDEV	Requests per minute for EDEV
REQUESTS_PER_MINUTE_TM	Requests per minute for TM
REQUESTS_PER_MINUTE_DCAP	Requests per minute for DCAP
REQUESTS_PER_MINUTE_DERP	Requests per minute for DERP
REQUESTS_BURST	Request burst limit
REQUEST_SIZE_KB	Request size limit in KB
ALLOCATED_MEM_MB	Allocated memory in MB

4.2.4. LWTS Manager Configuration

Update the `lwts-manager` environment variable:

```
DEFAULT_NETWORK_ID: FI
```

4.2.5. Download Kafka CA Certificate

```
kubectl get secret markets-kafka-cluster-ca-cert -n foundation-env-default \
  -o jsonpath='{.data.ca\.crt}' | base64 -d > ca.crt
```

4.2.6. Collect Module Client Credentials

Generic Command

```
echo "clientId: $(kubectl get secret <client-id> -n <module-namespace> \
  -o jsonpath='{.data.clientId}' | base64 -d)
clientSecret: $(kubectl get secret <client-id> -n <module-namespace> \
  -o jsonpath='{.data.clientSecret}' | base64 -d)
PROJECT_ID: $(kubectl get configmap -n foundation-cluster-zerotrust \
  zitadel-projects -o jsonpath='{.data.foundation\.foundation\.id}')"
```

Gateway Module Example

```
echo "clientId: $(kubectl get secret dergateway-app-admin -n ieee2030 \
  -o jsonpath='{.data.clientId}' | base64 -d)
```

```
clientSecret: $(kubectl get secret dergateway-app-admin -n ieee2030 \
  -o jsonpath='{.data.clientSecret}' | base64 -d)
PROJECT_ID: $(kubectl get configmap -n foundation-cluster-zerotrust \
  zitadel-projects -o jsonpath='{.data.foundation\.foundation\.id}')"
```

4.3. PTF Deployment

4.3.1. Create Persistent Volume Directories

Connect to the PTF Kubernetes cluster node and run:

```
sudo -i
```

Then execute the following script:

```
#!/bin/bash

# Directories to manage
DIRS=(
    "/root/generated_files/master"
    "/root/generated_files/worker"
)

for dir in "${DIRS[@]}"; do
    if [ -d "$dir" ]; then
        echo "Directory $dir exists. Deleting..."
        rm -rf "$dir"
    fi

    echo "Creating $dir..."
    mkdir -p "$dir"
done

echo "All directories have been processed."
```

4.3.2. Connect to PTF Cluster

```
export KUBECONFIG=ptf-cluster.yaml
```

4.3.3. Verify Cluster Connection

```
kubectl config view
```

4.3.4. Create Namespace for PTF

```
kubectl create namespace <ptf-namespace-name>
```

4.3.5. Create Kafka CA Secret

```
kubectl create secret generic kafka-ca --from-file=ca.crt=<cert-path>/ca.crt --namespace <ptf-
namespace-name>
```

> **Important**
>
> Use the `ca.crt` file downloaded in the pre-deployment steps.

4.3.6. Create Configuration ConfigMap

```
kubectl create configmap ptf-config --from-file=<config_file_path> --namespace <ptf-namespace-name>
```

4.3.7. Verify ConfigMap Creation

```
kubectl get configmap --namespace <ptf-namespace-name>
```

Expected output should include `ptf-config`.

4.3.8. Deploy PTF Helm Chart

```
helm install <release-name> <path-to-tgz-package> --values <path-to-values-file> --namespace <ptf-
namespace-name>
example: helm install ptf ptf-1.0.0.tgz --values values.yaml --namespace derms-ptf
```

4.4. Values File Configuration

Below is the complete values file used for this deployment:

```yaml
node:
  hostname: ip-172-31-4-224.ec2.internal

docker:
  registry: "<customer-artifactory-ptf-images-path>"
  tag: 1.0.0
  pullPolicy: Always
  initContainer:
```

```yaml
    image: "<customer-artifactory-busybox-image-path>"/busybox
    tag: 1.31.1

common:
  environment:
    gatewayHost: ""
    lfdiFileCreationWaitTimeSec: 30

kafka:
  environment:
    kafkaEnableSsl: true
  masterNodeIp: ""

zitadel:
  secret:
    projectId: ""
    gateway:
      gwClientId: ""
      gwClientSecret: ""
    operations:
      opsClientId: ""
      opsClientSecret: ""
  environment:
    useFoundationAuth: true
    zitadelBaseUrl: ""
    proxy: ""
    sslVerification: false

telemetry:
  environment:
    gatewayProxyHost: ""

locust:
  environment:
    headless: true
    locustHost: ""
    locustMode: distributed
    locustExpectWorkers: 5
    runTimeDuration: 4h
    ackWorkerTimeout: 20
    maxRetries: 5
    locustUsers: 50
    locustSpawnRate: 1
    workerWaitTime: 300

volumes:
  pv:
    localPathMaster: /root/generated_files/master
    localPathWorker: /root/generated_files/worker

imagePullSecret:
  secretName: registry-credentials
  registryName: ""
```

```
  username: "" # username for registry
  password: "" # password for registry

locustWebAuth:
  name: locust-web-auth
  enableWebLogin: false
  username: "" # username for locust web ui
  password: "" # password for locust web ui
  flaskSecretKey: "" # secret key for flask session management
```

4.5. Verification Steps

After deployment, verify the following:

4.5.1. Pod Status

```
kubectl get pods -l app=performance-test-framework --namespace <ptf-namespace-name>
```

4.5.2. Service Status

```
kubectl get services --namespace <ptf-namespace-name>
```

4.5.3. ConfigMap and Secrets

```
kubectl get configmaps --namespace <ptf-namespace-name>
kubectl get secrets --namespace <ptf-namespace-name>
```

4.5.4. Logs

```
kubectl logs -l app=locust-master --namespace <ptf-namespace-name>
kubectl logs -l app=locust-worker --namespace <ptf-namespace-name>
```

4.6. Troubleshooting

4.6.1. Common Issues

ImagePullBackOff

- Verify registry credentials are correct.
- Check network connectivity to Artifactory.
- Ensure the specified image tag exists.

Persistent Volume Issues

- Verify directories were created with proper permissions.
- Check node filesystem space.
- Ensure paths match the values file configuration.

Authentication Failures

- Verify Zitadel client credentials.
- Check PROJECT_ID is correct.
- Ensure RBAC policies have been updated.

Network Connectivity

- Verify all required ports are open.
- Check DNS resolution for external services.
- Test connectivity between PTF and DERMS clusters.

4.7. Security Considerations

This value file contains sensitive information including:

- Client secrets for Zitadel authentication
- Registry credentials
- Flask secret keys

Ensure these credentials are:

- Stored securely
- Rotated regularly
- Not committed to version control
- Accessed only by authorized personnel

4.8. Related Documentation

DERMS Performance Test Framework - Environment Variables

5. Data Generation Service - Accessing Generated Data

5.1. Overview

The DERMS Performance Test Framework includes a data generation service that creates test data for performance testing scenarios. This chapter explains how to access the generated data files stored on Kubernetes nodes, with different locations for master and worker nodes.

5.2. Prerequisites

Before accessing generated data, ensure you have:

- SSH access to Kubernetes cluster nodes.
- Appropriate permissions to access node filesystems.
- Knowledge of your cluster's node architecture (master versus worker nodes).
- Understanding of your PTF data generation configuration.

5.3. Data Storage Locations

The data generation service stores files in different locations on the node:

Node Type	Directory Path	Description
Master	`/root/generated_files/master`	Contains master service specific generated data.
Worker Node	`/generated_files/worker`	Contains worker service specific generated data by worker unique ID.

5.4. Accessing Master Node Data

5.4.1. Step 1: Connect to Master Node

Connect to your Kubernetes master node via SSH:

```
ssh root@<master-node-ip>
```

Or, if using a specific SSH key:

```
ssh -i /path/to/ssh-key root@<ptf-cluster-k8-node-ip>
```

5.4.2. Step 2: Navigate to Master Data Directory

Once connected to the master node, navigate to the generated files directory:

```
cd /root/generated_files/master
```

5.4.3. Step 3: List Generated Files

View all generated data files:

```
ls -la
```

Example output might include:

```
drwxr-xr-x 3 root root 4096 Oct  1 14:30 .
drwxr-xr-x 4 root root 4096 Oct  1 14:25 ..
drwxr-xr-x 2 root root 1024000 Oct  1 14:30 partitions
-rw-r--r-- 1 root root 2048000 Oct  1 14:30 ders_info.csv
-rw-r--r-- 1 root root 512000 Oct  1 14:30 FI.json
```

5.5. Accessing Worker Node Data

5.5.1. Step 1: Navigate to Worker Data Directory

Navigate to the worker's generated files directory:

```
cd /generated_files/worker
```

5.5.2. Step 2: List Generated Worker Directories

View the worker directories organized by unique worker IDs:

```
ls -lah
```

Example worker node output showing scaled workers with unique identifiers:

```
total 20K
drwxr-xr-x 5 root root 4.0K Sep 30 16:39 .
drwxrwxr-x 4 vini vini 4.0K Sep 30 16:38 ..
drwxr-xr-x 3 root root 4.0K Sep 30 16:39 0f2057588846_5d7c268d6b354573933a4c8ec6887b91
drwxr-xr-x 3 root root 4.0K Sep 30 16:39 51b10c3aff5f_7ece338b1dab46008544e1f124925af1
drwxr-xr-x 3 root root 4.0K Sep 30 16:39 b5c43cc09fe4_38edbde41c2040a7bb0d59524e0ea4ae
```

5.5.3. Step 3: Access Individual Worker Data

Each worker directory contains data specific to that scaled worker instance. Navigate to a specific worker directory:

```
ls <worker-unique-id>/
```

Example content of a worker directory:

```
ls 0f2057588846_5d7c268d6b354573933a4c8ec6887b91/
```

```
0f2057588846_5d7c268d6b354573933a4c8ec6887b91.csv  aggregators  lfdi_list.csv
```

The worker directory typically contains:

- `<worker-id>.csv`: Worker-specific DER data file.
- `aggregators/`: Directory containing aggregator-related data.
- `lfdi_list.csv`: List of aggregator identifiers.

5.6. Data Types and Formats

The data generation service typically creates various types of files:

- **CSV Data Files:** Contains DERs information with structured data.
- **JSON Data Files:** Contains LNM data files.

5.7. Copying Data Files

5.7.1. Copy Files to Local Machine

To copy generated files from nodes to your local machine:

From PTF Cluster Node

```
scp root@<ptf-cluster-k8-node-ip>:/root/generated_files/master/* ./master-data/
```

5.8. Best Practices

5.8.1. Data Access

- Always backup important generated data before modifications.
- Use appropriate file permissions to secure sensitive test data.
- Monitor disk space regularly to prevent storage issues.

5.8.2. Security

- Ensure SSH keys are properly secured.
- Limit access to generated data directories.
- Regularly audit file access patterns.

5.9. Integration with PTF Workflows

The generated data can be used for:

- Load testing scenarios.
- Performance benchmarking.

> **Tip**
>
> Set up automated scripts to regularly collect and analyze generated data for continuous performance monitoring.

6. Log Analysis in Headless Mode

6.1. Overview

This chapter explains how to analyze DERMS Performance Test Framework (PTF) logs when the framework is running in headless mode. In headless mode, DERMS Performance Test Framework (PTF) executes tests without a graphical interface, and the logs are stored within Kubernetes worker pods. These logs must be accessed directly from the pods for analysis.

6.2. Prerequisites

Before proceeding with log analysis, ensure you have:

- Access to the Kubernetes cluster where PTF is deployed.
- `kubectl` command-line tool configured and authenticated.
- Appropriate permissions to access pods and logs in the PTF namespace.
- Basic knowledge of Kubernetes pod management.

6.3. Accessing Kubernetes Worker Pods

6.3.1. Step 1: List PTF Worker Pods

Identify the PTF worker pods in your Kubernetes cluster:

```
kubectl get pods -n <ptf-namespace> -l app=ptf-worker
```

Example Output:

```
NAME                          READY   STATUS    RESTARTS   AGE
ptf-worker-1-abc123           1/1     Running   0          2h
ptf-worker-2-def456           1/1     Running   0          2h
ptf-worker-3-ghi789           1/1     Running   0          2h
```

6.3.2. Step 2: Connect to Worker Pod

To access a specific worker pod for log analysis, use the following command:

```
kubectl exec -it <pod-name> -n <ptf-namespace> -- /bin/bash
```

Example:

```
kubectl exec -it ptf-worker-1-abc123 -n ptf-namespace -- /bin/bash
```

6.4. Accessing Log Files

6.4.1. Navigate to Logs Directory

Once connected to the worker pod, navigate to the PTF logs directory:

```
cd /performance-test-framework/logs
```

6.4.2. List Available Log Files

View all available log files in the directory:

```
ls -la
```

Common log files you might find:

- `info.log`: High-level information
- `error.log`: Error and exception logs
- `debug.log`: Detailed debug information
- `critical.log`: Critical and exception logs

6.5. Log Analysis Commands

6.5.1. Viewing Log Contents

View Entire Log File

```
cat ptf-execution.log
```

View Last N Lines

```
tail -n 100 ptf-execution.log
```

View First N Lines

```
head -n 50 ptf-execution.log
```

Follow Log in Real-time

```
tail -f ptf-execution.log
```

6.5.2. Searching Log Contents

Search for Specific Terms

```
grep "ERROR" ptf-execution.log
grep -i "failed" *.log
```

Search with Context Lines

```
grep -A 5 -B 5 "EXCEPTION" ptf-execution.log
```

Count Occurrences

```
grep -c "WARNING" ptf-execution.log
```

6.5.3. Filtering and Analysis

Filter by Timestamp

```
grep "2025-10-01 14:" ptf-execution.log
```

Extract Performance Metrics

```
grep "response_time\|throughput\|latency" performance-metrics.log
```

Find Test Results

```
grep -E "(PASS|FAIL|SKIP)" test-results.log
```

6.6. Copying Logs for External Analysis

Copy a single file from Pod

To copy a log file from the pod to your local machine:

```
kubectl cp <pod-name>:/performance-test-framework/logs/ptf-execution.log ./ptf-execution.log -n <ptf-
namespace>
```

Copy entire logs directory

To copy all log files:

```
kubectl cp <pod-name>:/performance-test-framework/logs ./ptf-logs -n <ptf-namespace>
```

6.7. Log Analysis Best Practices

6.7.1. Performance Analysis

- **Identify bottlenecks**: Look for response time spikes or throughput drops.
- **Check resource usage**: Monitor CPU, memory, and I/O metrics.
- **Analyze error patterns**: Group similar errors and identify root causes.

6.7.2. Error Investigation

1. **Start with error logs**: Begin analysis with `error.log` or grep for "ERROR".
2. **Check stack traces**: Look for complete exception stack traces.
3. **Correlate timestamps**: Match errors with specific test execution times.

6.7.3. Test Execution Analysis

- **Verify test completion**: Ensure all planned tests executed.
- **Check test duration**: Identify tests that took longer than expected.
- **Review Pass/Fail ratios**: Analyze overall test success rates.

6.8. Common Log Patterns and Solutions

High Response Times

```
2025-10-01 14:30:15 WARN Response time exceeded threshold: 5000ms
```

Solution: Check system resources and network connectivity.

Memory Issues

```
2025-10-01 14:25:30 ERROR OutOfMemoryError: Java heap space
```

Solution: Increase pod memory limits or optimize test parameters.

Connection Failures

```
2025-10-01 14:20:45 ERROR Connection refused to target system
```

Solution: Verify network connectivity and target system availability.

6.9. Monitoring Multiple Pods

For comprehensive analysis across multiple worker pods:

```
# Get logs from all PTF worker pods
for pod in $(kubectl get pods -n <ptf-namespace> -l app=ptf-worker -o name); do
    echo "=== Logs from $pod ==="
    kubectl logs $pod -n <ptf-namespace> --tail=100
done
```

6.10. Troubleshooting

6.10.1. Pod Access Issues

If you cannot access a pod:

1. **Verify pod status:** `kubectl describe pod <pod-name> -n <ptf-namespace>`.
2. **Check pod logs:** `kubectl logs <pod-name> -n <ptf-namespace>`.
3. Ensure proper RBAC permissions.

6.10.2. Missing Log Files

If log files are not found:

1. Check if PTF is configured for file logging.
2. Verify log directory permissions.
3. Ensure tests have been executed.

6.11. Additional Resources

- For real-time monitoring, consider using `kubectl logs -f <pod-name>`.
- Use log aggregation tools like ELK stack for centralized analysis.
- Set up alerts for critical error patterns in production environments.

> **Tip**
>
> Always preserve important log files by copying them to persistent storage before pods are terminated or restarted.

7. Locust Web UI Guide for DERMS Performance Test Framework

7.1. Overview

The DERMS Performance Test Framework (PTF) supports running load tests through Locust's web interface, which provides an interactive dashboard for configuring, monitoring, and controlling load tests. This chapter explains how to deploy and use the web UI mode instead of the headless mode.

7.2. Prerequisites

- Kubernetes cluster with Helm installed.
- Access to the PTF Helm chart.
- Ingress controller configured in your cluster.
- Valid SSL certificates (optional, for HTTPS).

7.3. Configuration

To enable Locust web UI mode, you need to configure specific values in your Helm chart deployment.

7.3.1. Required Configuration Values

7.3.1.1. Locust Web Authentication

Configure authentication settings for the web interface:

```yaml
locustWebAuth:
  name: locust-web-auth
  enableWebLogin: true
  username: "your-username"           # Replace with your desired username
  password: "your-secure-password"    # Replace with your secure password
  flaskSecretKey: "your-secret-key"   # Replace with a random secret key
```

> **Important**
>
> Replace the placeholder values with actual credentials and a secure secret key.

Proprietary - See Copyright Page

7.3.1.2. Ingress Configuration

Enable ingress to expose the Locust web UI:

```yaml
ingress:
  enabled: true
  host: service.ptf.derms.gridos.com
  name: locust-ingress
```

7.3.1.3. Locust Mode Configuration

Ensure Locust is configured for web UI mode (not headless):

```yaml
locust:
  environment:
    headless: false                    # Set to false for web UI mode
```

7.4. Deployment

7.4.1. Update Values File

Create or update your `values.yaml` file with the required configuration:

```bash
# Copy the existing values.yaml and modify it
cp helm/values.yaml my-values.yaml
```

Edit `my-values.yaml` to include the web UI configuration as shown above.

7.4.2. Deploy with Helm

Deploy the PTF with web UI enabled:

```bash
helm upgrade --install ptf-webui ./helm \
  --values my-values.yaml \
  --namespace ptf \
  --create-namespace
```

7.4.3. Verify Deployment

Check that all pods are running:

```bash
kubectl get pods -n ptf
```


Proprietary - See Copyright Page

Verify the ingress is created:

```
kubectl get ingress -n ptf
```

7.5. Accessing the Web UI

7.5.1. Access URL

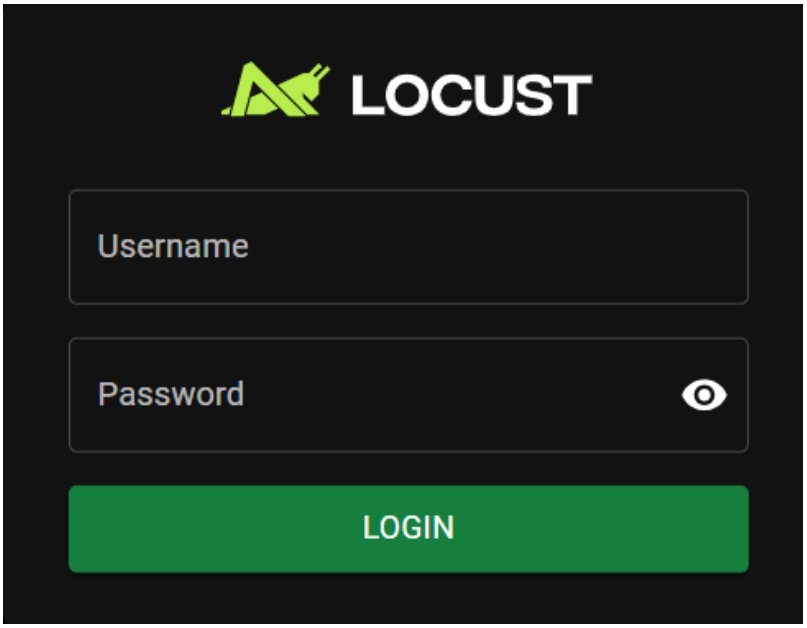
Once deployed, access the Locust web UI: https://service.ptf.derms.gridos.com/

> **Note**
>
> The URL uses the host configured in your ingress settings.

7.5.2. Login

Use the credentials configured in `locustWebAuth`:

- **Username**: The value you set for `locustWebAuth.username`
- **Password**: The value you set for `locustWebAuth.password`

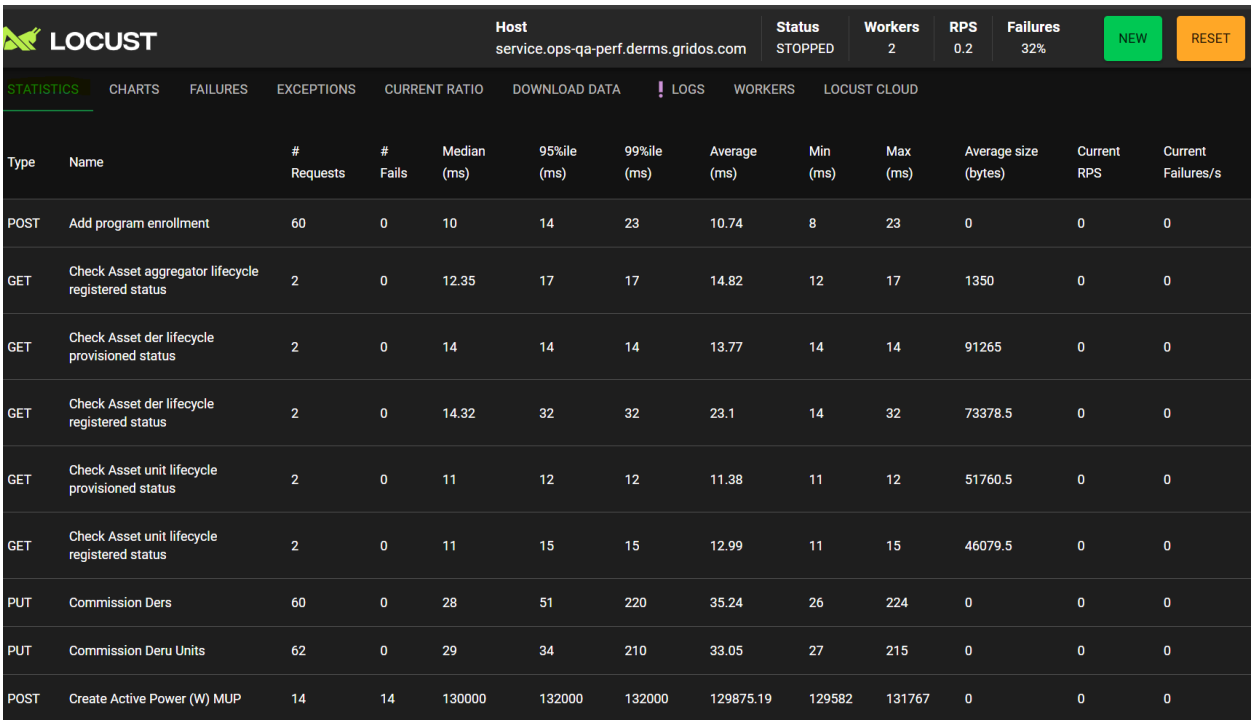


7.6. Using the Web UI

7.6.1. Dashboard Overview

After logging in, you'll see the main dashboard with:

- Test configuration options
- Real-time statistics
- Charts and graphs
- Worker status



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	Add program enrollment	60	0	10	14	23	10.74	8	23	0	0	0
GET	Check Asset aggregator lifecycle registered status	2	0	12.35	17	17	14.82	12	17	1350	0	0
GET	Check Asset der lifecycle provisioned status	2	0	14	14	14	13.77	14	14	91265	0	0
GET	Check Asset der lifecycle registered status	2	0	14.32	32	32	23.1	14	32	73378.5	0	0
GET	Check Asset unit lifecycle provisioned status	2	0	11	12	12	11.38	11	12	51760.5	0	0
GET	Check Asset unit lifecycle registered status	2	0	11	15	15	12.99	11	15	46079.5	0	0
PUT	Commission Ders	60	0	28	51	220	35.24	26	224	0	0	0
PUT	Commission Deru Units	62	0	29	34	210	33.05	27	215	0	0	0
POST	Create Active Power (W) MUP	14	14	130000	132000	132000	129875.19	129582	131767	0	0	0

Figure 1. Main Dashboard Overview

7.6.2. Starting a Load Test

To start a load test:

1. Configure Test Parameters:
    - Number of users (total user count)
    - Spawn rate (users per second)
    - Host URL (target system)
2. Start test:

- Click **Start swarming**
- Monitor real-time statistics

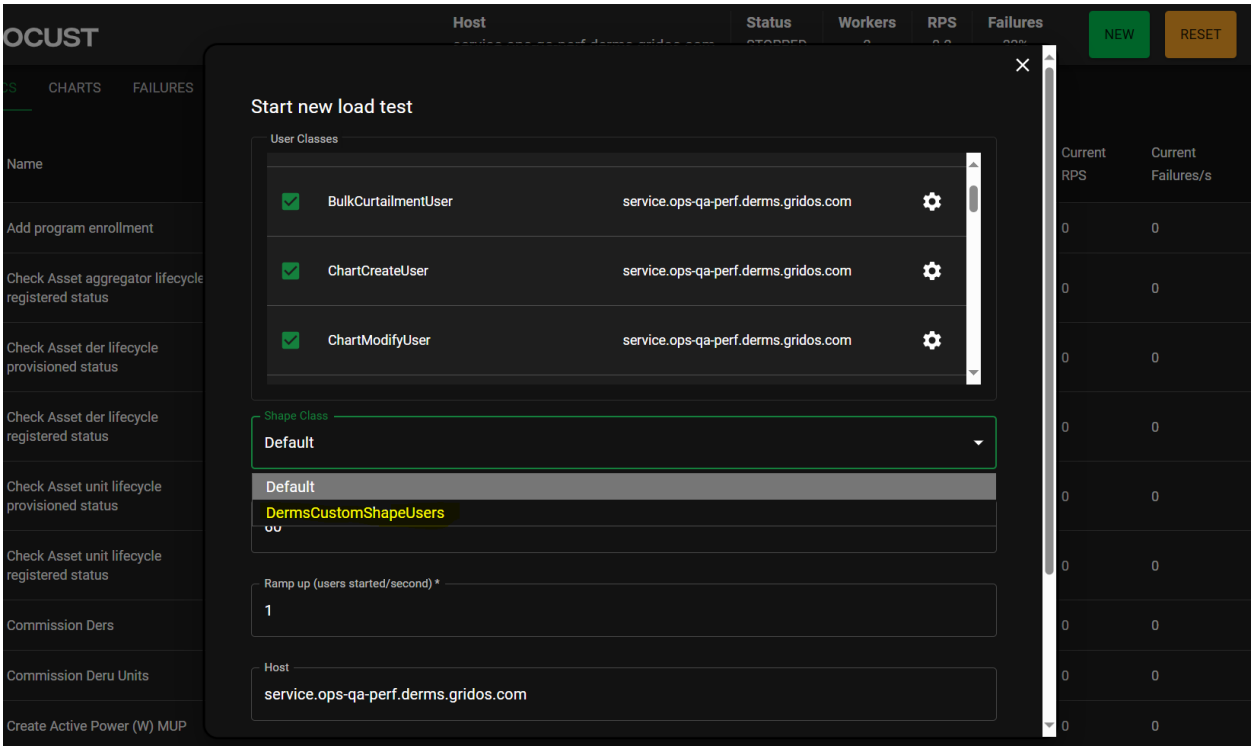


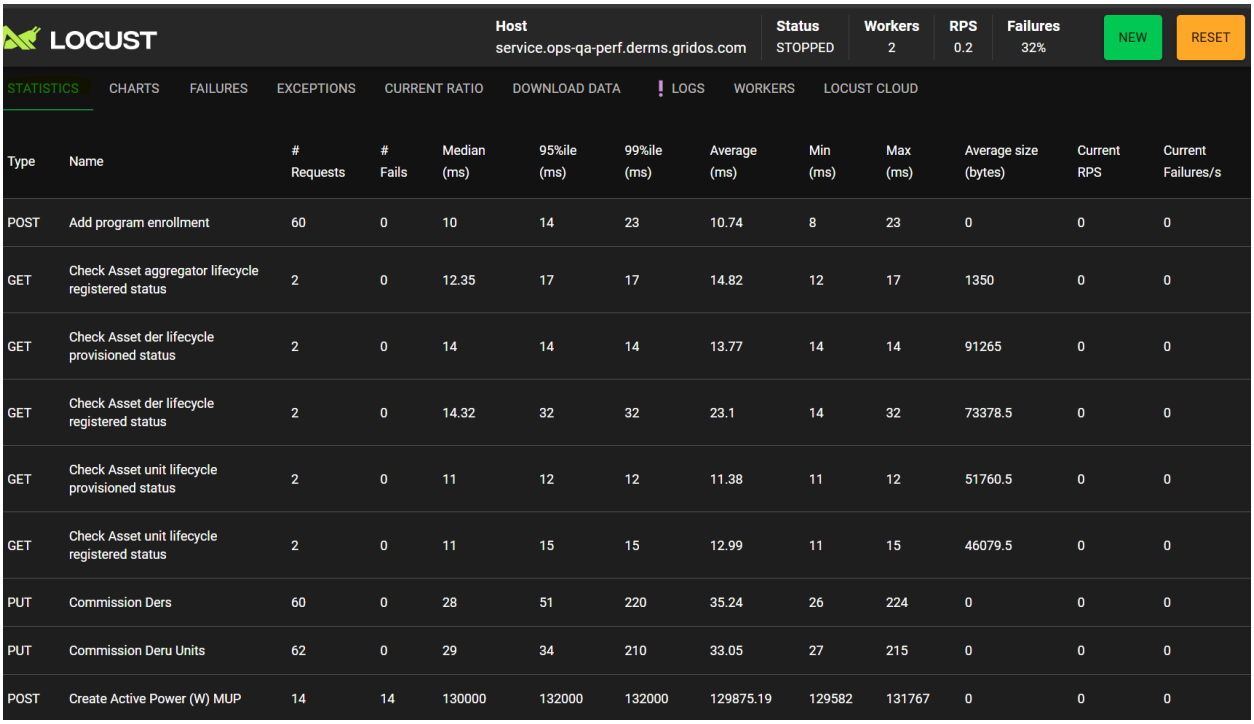
Figure 2. Load Test Configuration Panel

7.6.3. Monitoring Test Results

The web UI provides several monitoring views:

7.6.3.1. Statistics Tab

- Request statistics
- Response times
- Failure rates
- RPS (Requests per second)



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	Add program enrollment	60	0	10	14	23	10.74	8	23	0	0	0
GET	Check Asset aggregator lifecycle registered status	2	0	12.35	17	17	14.82	12	17	1350	0	0
GET	Check Asset der lifecycle provisioned status	2	0	14	14	14	13.77	14	14	91265	0	0
GET	Check Asset der lifecycle registered status	2	0	14.32	32	32	23.1	14	32	73378.5	0	0
GET	Check Asset unit lifecycle provisioned status	2	0	11	12	12	11.38	11	12	51760.5	0	0
GET	Check Asset unit lifecycle registered status	2	0	11	15	15	12.99	11	15	46079.5	0	0
PUT	Commission Ders	60	0	28	51	220	35.24	26	224	0	0	0
PUT	Commission Deru Units	62	0	29	34	210	33.05	27	215	0	0	0
POST	Create Active Power (W) MUP	14	14	130000	132000	132000	129875.19	129582	131767	0	0	0

Figure 3. Statistics Tab with Real-time Metrics

7.6.3.2. Charts Tab

- Response time charts
- RPS over time
- User count progression

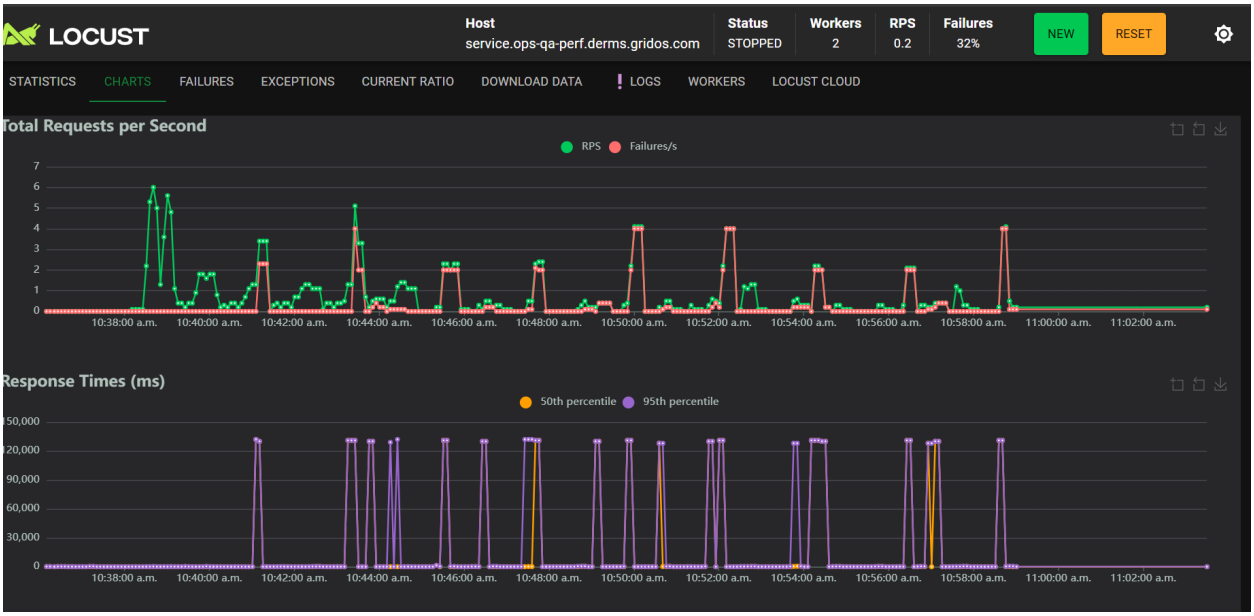


Figure 4. Charts Tab showing Performance Metrics

7.6.3.3. Failures Tab

- Detailed failure information
- Error messages and counts
- Failure distribution

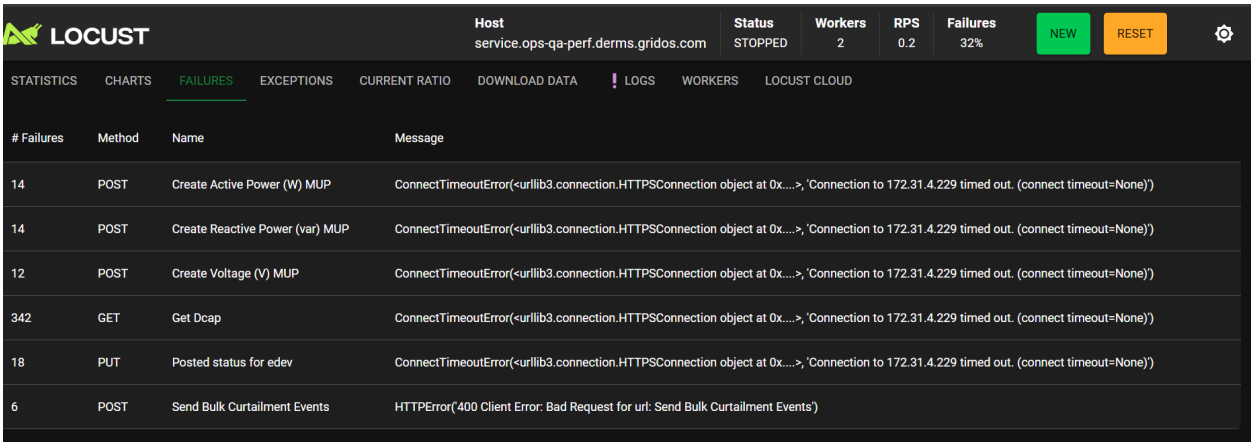


Figure 5. Failure Tab with Error Details

7.6.4. Download Test Data

During test execution, you can:

- **Download Data**: Export results as CSV

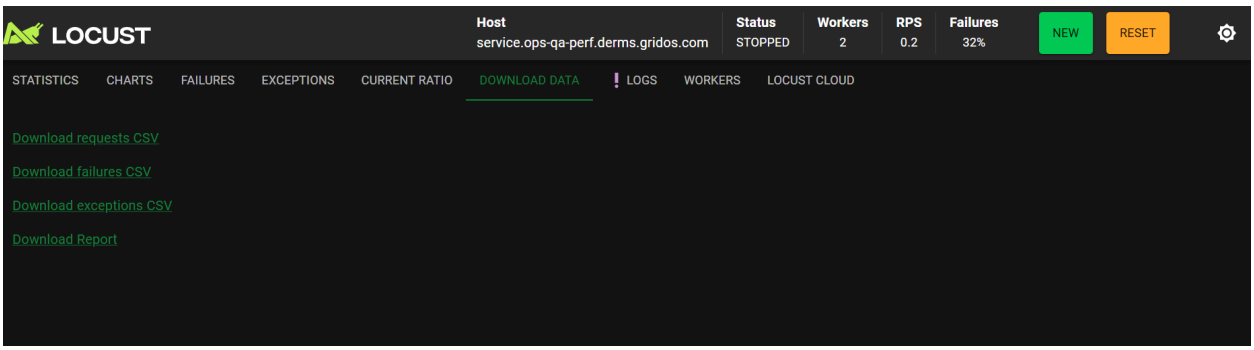
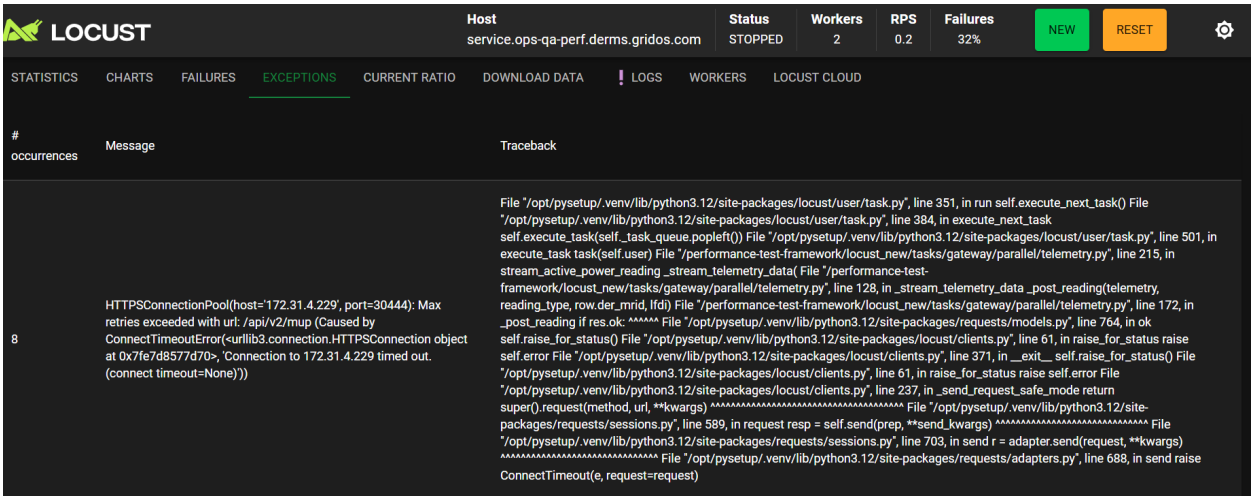


Figure 6. Test Download Report

7.6.5. Exceptions PTF

During test execution, if you see any exceptions in the web UI, you can:

- **Exceptions in PTF**: Displays application exceptions


LOCUST
Host
service.ops-qa-perf.derms.gridos.com
Status
STOPPED
Workers
2
RPS
0.2
Failures
32%
NEW
RESET
STATISTICS
CHARTS
FAILURES
EXCEPTIONS
CURRENT RATIO
DOWNLOAD DATA
LOGS
WORKERS
LOCUST CLOUD
occurrences
Message
Traceback
8
HTTPSConnectionPool(host='172.31.4.229', port=30444): Max retries exceeded with url: /api/v2/mup (Caused by ConnectTimeoutError(<urllib3.connection.HTTPSConnection object at 0x7fe7d8577d70>, 'Connection to 172.31.4.229 timed out. (connect timeout=None)'))
File "/opt/pysetup/.venv/lib/python3.12/site-packages/locust/user/task.py", line 351, in run self.execute_next_task() File "/opt/pysetup/.venv/lib/python3.12/site-packages/locust/user/task.py", line 384, in execute_next_task self.execute_task(self._task_queue.popleft()) File "/opt/pysetup/.venv/lib/python3.12/site-packages/locust/user/task.py", line 501, in execute_task task(self.user) File "/performance-test-framework/locust_new/tasks/gateway/parallel/telemetry.py", line 215, in stream_active_power_reading _stream_telemetry_data(File "/performance-test-framework/locust_new/tasks/gateway/parallel/telemetry.py", line 128, in _stream_telemetry_data _post_reading(telemetry, reading_type, row.der_mrid, lfdi) File "/performance-test-framework/locust_new/tasks/gateway/parallel/telemetry.py", line 172, in _post_reading if res.ok: ^^^^^^ File "/opt/pysetup/.venv/lib/python3.12/site-packages/requests/models.py", line 764, in ok self.raise_for_status() File "/opt/pysetup/.venv/lib/python3.12/site-packages/locust/clients.py", line 61, in raise_for_status raise self.error File "/opt/pysetup/.venv/lib/python3.12/site-packages/locust/clients.py", line 371, in __exit__ self.raise_for_status() File "/opt/pysetup/.venv/lib/python3.12/site-packages/locust/clients.py", line 61, in raise_for_status raise self.error File "/opt/pysetup/.venv/lib/python3.12/site-packages/locust/clients.py", line 237, in _send_request_safe_mode return super().request(method, url, **kwargs) ^^^^^^

- **Workers User Status**: Displays active load test users

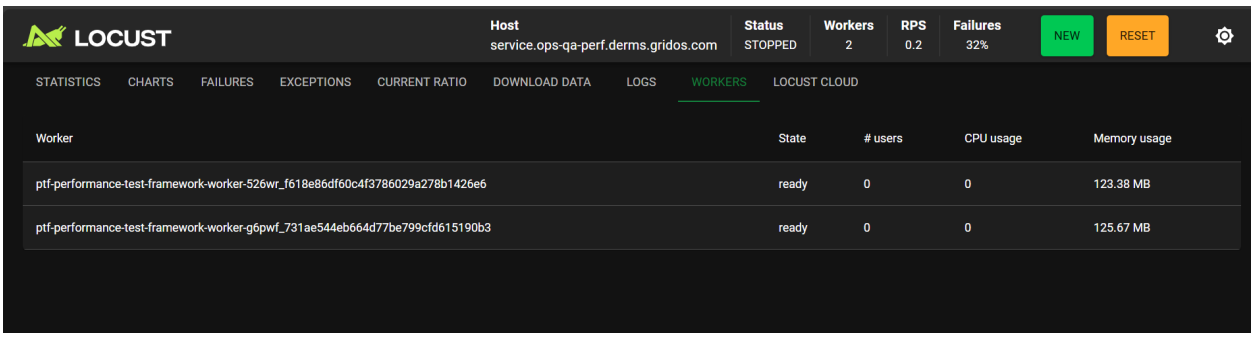


Figure 9. Load Test Active Users States in Worker

7.7. Troubleshooting

7.7.1. Common Issues

7.7.1.1. Web UI Not Accessible

1. Check Ingress Status:

```
kubectl describe ingress locust-ingress -n ptf
```

2. Verify Service:

```
kubectl get svc -n ptf
```

3. Check Pod Logs:

```
kubectl logs -l app=locust-master -n ptf
```

7.7.2. Authentication Issues

1. Verify the secret is created:

```
kubectl get secret locust-web-auth -n ptf
```

2. Check if credentials are correct in the values file

7.7.3. Workers Not Connecting

1. Check worker pod logs:

```
kubectl logs -l app=locust-worker -n ptf
```

2. Verify master service is accessible:

```
kubectl get svc locust-master -n ptf
```

7.7.4. Debug Mode

Enable debug logging by adding to your values file:

```
locust:
  environment:
    logLevel: DEBUG
```

7.8. Best Practices

7.8.1. Security

- Use strong passwords for web UI authentication.
- Generate secure Flask secret keys.
- Consider using HTTPS in production.
- Restrict ingress access using network policies.

7.8.2. Performance

- Monitor cluster resources during tests.
- Scale workers based on target load.
- Use appropriate spawn rates to avoid overwhelming the system.

7.8.3. Testing Strategy

- Start with small user counts to validate setup.
- Gradually increase load to find system limits.
- Use realistic test data and scenarios.

Proprietary - See Copyright Page

- Monitor both client and server metrics.

7.9. Conclusion

The Locust web UI provides a powerful interface for interactive load testing with real-time monitoring and control capabilities. By following this chapter, you can successfully deploy and use the web UI mode for your performance testing needs.

For additional support or questions, refer to the project documentation or contact the development team.